School of Future Tech

Case Study Report

on

Transport Billing & Invoice Management System

by

Ayush Kumar Singh

1500967250

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Transport companies typically manage billing through manual processes such as handwritten receipts or spreadsheets. These methods are time-consuming, prone to calculation errors, and difficult to track as trip volumes grow. This case study focuses on designing and implementing a **Transport Billing & Invoice Management System** using C++ that automates the full billing lifecycle.

The system is built using core Object-Oriented Programming (OOP) principles in C++, covering client registration, trip logging, invoice generation with 18% GST, payment processing, and saving invoices to text files. It demonstrates how academic concepts like classes, vectors, file handling, and input validation apply to a real-world business problem.

2. Problem Statement / Case Background (Abstract)

Background

Manual billing in transport operations leads to frequent errors in fare calculation, missing payment records, no GST tracking, and difficulty retrieving past invoices. There is a clear need for a simple automated system to handle these tasks reliably and consistently.

Abstract

This case study presents the design and implementation of a **Transport Billing & Invoice Management System** using C++. The system allows operators to register clients, log transport trips, generate itemised invoices with 18% GST, process payments with change calculation, and save invoices as persistent text files. It uses four core classes — Client, Trip, Invoice, and Payment — stored in STL vectors. Input validation prevents crashes on bad input and file handling ensures invoice data is retained after the session.

3. Case Study Design

The system is designed as a menu-driven C++ console application. Each class handles one responsibility:

1. **Client** — Stores client ID, name, and contact number.
2. **Trip** — Stores trip details and calculates subtotal and 18% GST.
3. **Invoice** — Links a Client and Trip; prints and saves the invoice to a .txt file.
4. **Payment** — Processes payment, calculates change, and marks the invoice as PAID.
5. **getNumber()** — Helper function for input validation; re-prompts on bad input.
6. **main()** — Menu loop, vector storage, and memory management.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Key methods and algorithms:

1. **Billing calculation**
    - Subtotal = baseFare + (distanceKm x ratePerKm) + extraCharges.
    - GST = Subtotal x 0.18.
    - Total = Subtotal + GST.
2. **OOP concepts applied**
    - Encapsulation: all data members private, accessed via const getters.
    - Abstraction: billing logic encapsulated inside Trip::calculateTotalCost().
    - Pointers and dynamic memory: objects created with new, deleted at exit.
3. **Input validation**
    - getNumber() helper uses cin.fail() and cin.clear() to handle non-numeric input.
    - Menu choice validated with cin.fail() check before processing.

Technology Stack Used for the Case

- Programming language: C++ (C++17).
- Libraries and tools:
    - <iostream> for console input and output.
    - <fstream> for writing invoice files using ofstream.
    - <vector> for dynamic storage of object pointers.
    - <iomanip> for formatted currency output with setprecision(2).
- Compiler: g++ with C++17 standard flag.
- IDE: VS Code, Code::Blocks, or any C++ IDE.

5. Problem Statement / Case Study Implementation Details and Snapshots.

The system is implemented in a single C++ source file with five menu-driven operations:

1. **Register New Client**
    - User enters name and contact. A Client object is created with an auto-incremented ID starting at 101 and stored in the clients vector.
2. **Log New Transport Trip**
    - Distance, rate per km, base fare, and extra charges are entered via getNumber(). A Trip object is created and stored.
3. **Generate Invoice**
    - An Invoice object links the latest Client and Trip. Total is auto-calculated with 18% GST. Invoice is printed to console and saved as Invoice_<ID>.txt.
4. **Process Payment**

    - Payment is processed; invoice is marked PAID if sufficient, change shown if overpaid. Invoice file is re-saved to reflect the updated status.
5. **View All Invoices**

Snapshots (attach below):

- Main menu screen showing all 6 options.

```
**************************************************
* Transport Billing & Invoice Management System   *
**************************************************


--- MAIN MENU ---
1. Register New Client
2. Log New Transport Trip
3. Generate Invoice
4. Process Payment
5. View All Invoices
6. Exit
Enter your choice:
```

- Invoice generated on console with GST breakdown.

```
Available Clients:
  [ID: 101]  Ayush  |  1234567812
Select Client ID: 101

Available Trips:
  [ID: 501]  100.00 km  @  Rs.30.00/km  |  Base: Rs.120.00  |  Extra: Rs.30.00  =>  Total: Rs.3717.00
Select Trip ID: 501

========================================
               INVOICE #1001
========================================
Client Details:
Name:    Ayush
Contact: 1234567812
----------------------------------------
Trip Details (Trip ID: 501):
Distance:       100.00 km
Rate per km:   Rs.30.00
Base Fare:     Rs.120.00
Extra Charges: Rs.30.00
----------------------------------------
Subtotal:      Rs.3150.00
GST (18%):     Rs.567.00
----------------------------------------
TOTAL AMOUNT:  Rs.3717.00
STATUS:        PENDING
========================================
[INFO] Invoice saved to Invoice_1001.txt
```

- Payment processed successfully with change displayed.

```
Enter your choice: 4

Pending Invoices:
  [ID: 1001]  Rs.3717.00  -  PENDING
Enter Invoice ID to pay: 1001
Total Due: Rs.3717.00
Enter payment amount: Rs.4000

[SUCCESS] Payment of Rs.4000.00 processed successfully.
[INFO] Change to return: Rs.283.00
[INFO] Invoice saved to Invoice_1001.txt
```

- Saved Invoice_1001.txt file showing STATUS: PAID after payment.

```
Enter your choice: 5

==========================================
                INVOICE #1001
==========================================
Client Details:
Name:    Ayush
Contact: 1234567812
------------------------------------------
Trip Details (Trip ID: 501):
Distance:        100.00 km
Rate per km:    Rs.30.00
Base Fare:      Rs.120.00
Extra Charges: Rs.30.00
------------------------------------------
Subtotal:       Rs.3150.00
GST (18%):      Rs.567.00
------------------------------------------
TOTAL AMOUNT:   Rs.3717.00
STATUS:         PAID
==========================================
```

6. Problem Statement / Case Study Results and Conclusion.

Findings

- Input validation with getNumber() completely prevents crashes from invalid input.
- GST calculation is accurate and the invoice breakdown clearly separates subtotal, GST, and total.
- File re-save after payment ensures the .txt file always reflects the true current payment status.

Conclusion

This case study demonstrates a complete C++ implementation of a Transport Billing and Invoice Management System. It integrates client registration, trip logging, billing calculation with GST, invoice file handling, and payment tracking into a coherent console application, providing a strong foundation for further development with database integration or a graphical interface.

7. References

- Bjarne Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley, 2013.
- cppreference.com — Official C++ standard library documentation including iostream, fstream, and vector.
- GeeksforGeeks — Tutorials on C++ OOP, file handling, vectors, and input validation.
- Government of India — GST Rate Structure, Ministry of Finance (https://www.gst.gov.in).
- ITM Skills University — B.Tech CSE Semester II Case Study Guidelines, 2025.